ER Model Part 1 – Terminology Concepts Reference

Database Systems

Based on Lecture 5 by Tiantian Liu

Fall 2025

Contents

1 Quick Reference Guide

ER Model Essentials - At a Glance

Basic Components:

- **Entity Type**: Rectangle
- **Weak Entity Type**: Double rectangle
- **Relationship Type**: Diamond
- **Weak Relationship Type**: Double diamond
- **Attribute**: Oval
- **Primary Key**: Underlined attribute
- **Discriminator**: Dashed underline

Cardinality Notations:

- **Graphical**: Arrow (→) = one, Line (—) = many
- **Chen**: 1, N, M labels on lines
- **Min/Max**: [min, max] on lines

Participation:

- **Total**: Double line (every entity must participate)
- **Partial**: Single line (some entities may not participate)

Key Relationship Patterns:

- 1:1 – Each entity relates to at most one entity
- 1:N – One entity can relate to many, but each related entity connects to at most one
- N:1 – Many entities can relate to one
- M:N – Many-to-many, no restrictions

2 Data Modelling Process

The Four Steps of Database Design

Database design follows a systematic four-step process:

1. Requirement Collection and Analysis

- Gather requirements from stakeholders
- Identify objects (entities) and their properties (attributes)
- Identify relationships between objects
- Specify constraints and business rules

2. **Conceptual Design**
   - Create ER diagram from requirements
   - Model entities, relationships, and attributes
   - Define cardinalities and participation constraints
   - Independent of specific DBMS
3. **Data Model Mapping**
   - Convert ER diagram to relational schema
   - Define tables, columns, and relationships
   - Specify primary keys and foreign keys
4. **Physical Design**
   - Implement schema in specific DBMS
   - Create indexes, optimize storage
   - Configure memory, pages, devices

Requirement Specification Example

For a university database, requirements might state:

- "The instructor ID unambiguously identifies an instructor"
- "Instructors are in different departments"
- "Instructors teach courses"
- "Students take courses"

Object Specification:

- Instructor: ID (char, not null, unique), name (varchar), dept_name (varchar), salary (numeric(8,2))
- Course: course_id (char, not null, unique), title (varchar), dept_name (varchar), credit (numeric(2,0))

Relationship Specification:

- Teaches: instructor ↔ course, attributes: semester, year
- Takes: student ↔ course, attributes: semester, year, grade

Finding Entities and Relationships: The Noun-Verb Heuristic

Requirement Statement	Entities (Nouns)	Relationships (Verbs)
"The **instructor** teaches a **course**"	instructor, course	teaches
"The **library** has many **books**"	library, book	has
"A **customer** rents a **video tape** of a **film**"	customer, video tape, film	rents
"**Students** take **courses**"	student, course	takes
"**Employees** work in **departments**"	employee, department	works_in

Concrete Data Examples: Understanding Cardinality Through Instances

Cardinality Type	Entity A Instances	Entity B Instances
1:N (One-to-Many)	**Instructor:** Einstein (ID=10101) Mozart (ID=12121) Brandt (ID=15151)	**Course:** Physics I Music History Finance, Comp Sci
	Interpretation: Each instructor teaches multiple courses, but each course is taught by at most one instructor	
M:N (Many-to-Many)	**Student:** Zhang (ID=00128) Shankar (ID=12345) Brown (ID=19991)	**Course:** Comp Sci, Finance Comp Sci, Music Finance
	Interpretation: Each student takes multiple courses, and each course has multiple students	
1:1 (One-to-One)	**Employee:** Smith (ID=501) Jones (ID=502) Davis (ID=503)	**Office:** Office 301 Office 302 Office 303
	Interpretation: Each employee has exactly one office, and each office is assigned to at most one employee	

Design Pitfalls to Avoid

Redundancy:

- Avoid repeating information in multiple places
- Can lead to data inconsistency
- Example: Storing department budget in both department and instructor tables

Incompleteness:

- Ensure all aspects of the enterprise can be modeled
- Don't omit critical relationships or attributes

- Example: Forgetting to model course prerequisites

3 ER Model Basics

3.1 Entity Types

Entity and Entity Type

Entity: An object that exists and is distinguishable from other objects.

- Examples: specific person, company, event, plant

Entity Set: A set of entities of the same type that share the same properties.

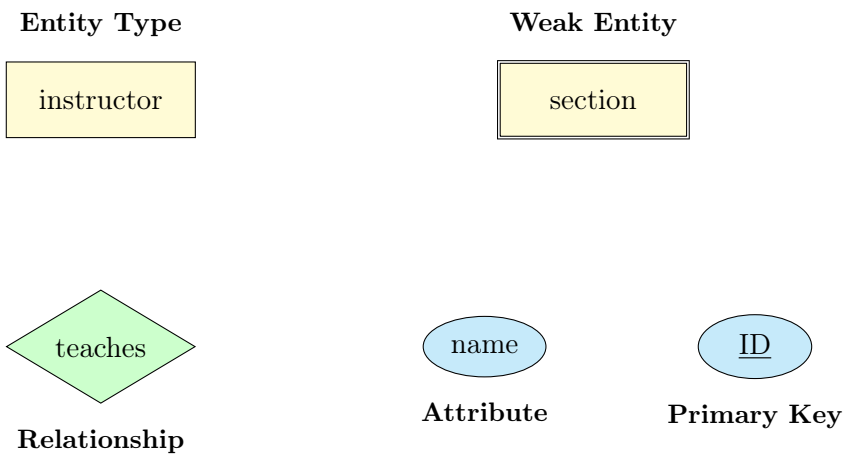
- Examples: set of all persons, companies, trees, holidays

Entity Type: Describes the structure of an entity set.

- Represented by a rectangle in ER diagrams
- Defines the attributes that all entities in the set possess

Attributes: Descriptive properties possessed by all members of an entity type.

- Example: instructor = (ID, name, salary)
- Example: course = (course_id, title, credits)



Finding Entities During Modeling

Heuristic: Look for nouns in the database requirements.
Examples:

- "The *instructor* teaches a *course*" → instructor, course are entities
- "The *library* has many *books*" → library, book are entities
- "A *customer* rents a *video tape* of a *film*" → customer, video tape, film are entities

3.2 Attributes

Attribute Types

1. Simple vs. Composite Attributes
Simple attribute: Cannot be divided into smaller parts.

- Example: phone number stored as a single string

Composite attribute: Can be divided into subparts with independent meaning.

- Example: address = (street, city, state, zip)
- Example: name = (first_name, middle_name, last_name)

2. Single-valued vs. Multi-valued Attributes
Single-valued: Has a single value for each entity.

- Example: date of birth, SSN

Multi-valued: Can have multiple values for each entity.

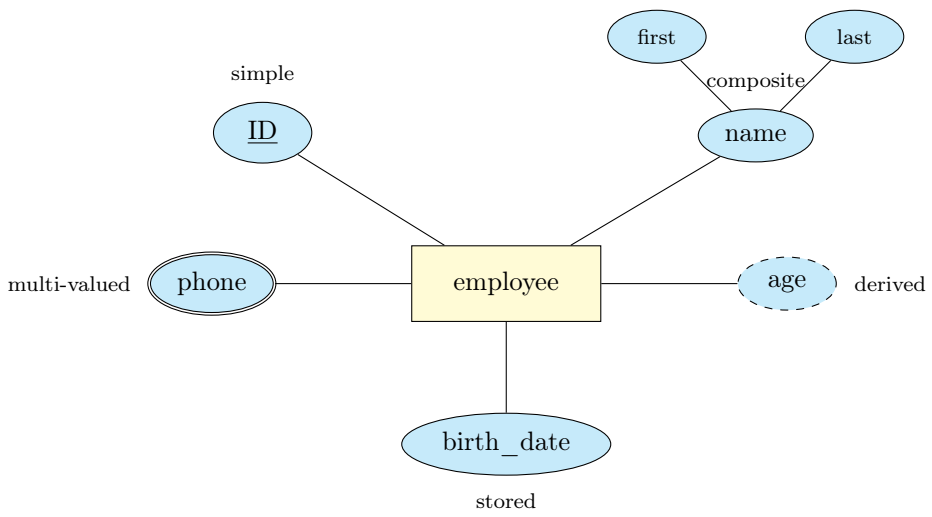
- Example: phone numbers (work, home, mobile)
- Example: email addresses
- Depicted with double oval in ER diagrams

3. Stored vs. Derived Attributes
Stored attribute: Value is explicitly stored in the database.

- Example: birth_date

Derived attribute: Value can be computed from other attributes.

- Example: age (derived from birth_date)
- Depicted with dashed oval in ER diagrams



Attribute Type Examples

Employee Entity:

- `employee_id`: Simple, single-valued, stored (primary key)
- `name`: Composite (first, middle, last), single-valued, stored
- `birth_date`: Simple, single-valued, stored
- `age`: Simple, single-valued, derived (from birth_date)
- `phone`: Simple, multi-valued, stored
- `address`: Composite (street, city, state, zip), single-valued, stored

Attribute Design Considerations

When to use composite attributes:

- If you need to query or manipulate parts independently
- Example: Search by city → use composite address

When to use derived attributes:

- Value changes automatically based on stored data
- Saves storage but increases computation
- Example: age changes every year, better to derive from birth_date

Multi-valued attributes:

- Often better modeled as separate entity with relationship
- Example: Multiple phone numbers → create phone_number entity

3.3 Primary Keys

Primary Key

A **primary key** is a set of attributes that uniquely identifies each entity in an entity set.
Properties:

- **Uniqueness**: No two entities can have the same primary key value
- **Minimality**: No proper subset of the key can uniquely identify entities
- **Non-null**: Primary key values cannot be NULL

Notation:

- In ER diagrams: underlined attribute
- Example: <u>ID</u>, name, salary

Primary Key Examples

Single-attribute keys:

- instructor(ID, name, dept_name, salary)
- course(course_id, title, dept_name, credits)
- student(ID, name, dept_name, tot_cred)

Composite keys:

- takes(student_ID, course_id, sec_id, semester, year, grade)
- section(course_id, sec_id, semester, year, building, room_number)

3.4 Relationships

Relationship and Relationship Type

Relationship: An association among several entities.

- Example: (student1, Computer Science) $\in$ takes

Relationship Set: A mathematical relation among $n \geq 2$ entities:

- $\{(e_1, e_2, \ldots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\}$
- Each $(e_1, e_2, \ldots, e_n)$ is a relationship instance

Relationship Type: Describes the structure of a relationship set.

- Represented by a diamond in ER diagrams
- Can have attributes (properties of the relationship itself)

Participation: Each entity set E_i participates in the relationship set.

Relationship Examples

Binary relationship (n=2):

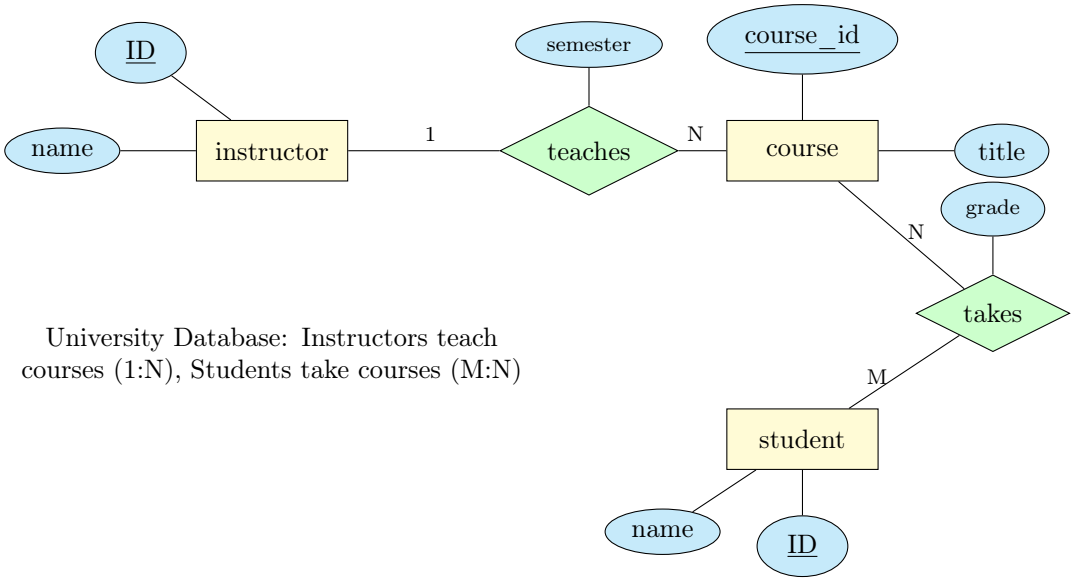
- `takes`: student $\leftrightarrow$ course
- Attributes: semester, year, grade

Ternary relationship (n=3):

- `proj_guide`: instructor $\leftrightarrow$ student $\leftrightarrow$ project
- Students work on projects under instructor guidance

Relationship with attributes:

- `teaches`: instructor $\leftrightarrow$ course
- Attributes: semester, year, section_id



University Database: Instructors teach courses (1:N), Students take courses (M:N)

Finding Relationships During Modeling

Heuristic: Look for verbs in the database requirements.
Examples:

- "Instructors *teach* courses" → teaches relationship
- "Students *take* courses" → takes relationship
- "Employees *work in* departments" → works_in relationship
- "Customers *rent* video tapes" → rents relationship

3.5 Roles

Roles in Relationships

Role: The function that an entity plays in a relationship.
When roles are needed:

- Entity sets in a relationship need not be distinct
- An entity type can participate multiple times
- Role names clarify the participation

Recursive Relationships:

- Relationships where the same entity type participates multiple times
- Role names are essential for clarity

Role Examples

Course prerequisites:

- Relationship: `prereq`
- Entity type: `course` (participates twice)

- Roles: `course_id` and `prereq_id`
- Meaning: course with course_id has prerequisite with prereq_id

Employee supervision:

- Relationship: `supervises`
- Entity type: `employee` (participates twice)
- Roles: `supervisor` and `supervisee`
- Meaning: one employee supervises another employee

Marriage (symmetric roles):

- Relationship: `married_to`
- Entity type: `person` (participates twice)
- Roles: `spouse1` and `spouse2`

3.6 Degree of Relationships

Relationship Degree

Degree: The number of entity types participating in a relationship.
Binary Relationship (Degree 2):

- Most common type
- Involves two entity types
- Example: instructor ↔ course

Ternary Relationship (Degree 3):

- Involves three entity types
- Example: instructor ↔ student ↔ project
- Use when relationship cannot be decomposed into binary relationships without loss of information

n-ary Relationship (Degree n):

- Involves n entity types
- Rare in practice (most relationships are binary)

When to Use Ternary Relationships

Use ternary when:

- The relationship fundamentally involves three entities
- Decomposing into binary relationships loses information

Example – Project Guidance:

- "Students work on projects under the guidance of an instructor"
- Cannot decompose into: (student, project) + (project, instructor)
- Why? Different students on same project may have different instructors
- Must use: (student, project, instructor)

Avoid ternary when:

- The relationship can be modeled with binary relationships without information loss
- Example: If each project has exactly one instructor, use binary relationships

4 Mapping Cardinality Constraints

Cardinality Constraints

Cardinality: Expresses the number of entities to which another entity can be associated via a relationship.
For binary relationships, four types:

1. **One-to-One (1:1)**: Each entity in A relates to at most one in B, and vice versa
2. **One-to-Many (1:N)**: Each entity in A can relate to many in B, but each in B relates to at most one in A
3. **Many-to-One (N:1)**: Many entities in A can relate to one in B
4. **Many-to-Many (M:N)**: Entities in A can relate to many in B, and vice versa

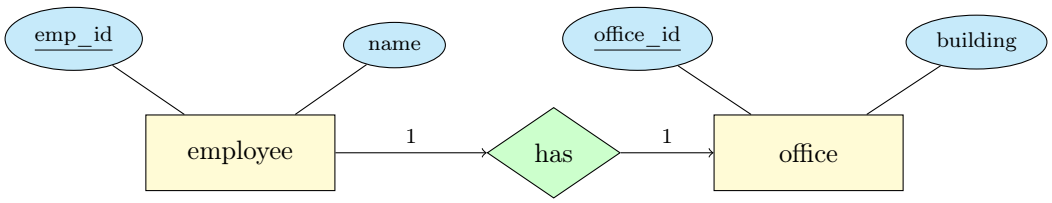
Note: Some elements may not be mapped (partial participation).

4.1 One-to-One (1:1)

One-to-One Relationship

Each entity in A is associated with **at most one** entity in B.
Each entity in B is associated with **at most one** entity in A.
Graphical Notation: Arrow on both sides (→←)
Chen Notation: 1 on both sides of the relationship diamond
[min, max] Notation: [0, 1] on both sides (or [1, 1] for total participation)

ER Diagram Example: Employee-Office (1:1 Relationship)



Plain English Explanation: Employee-Office (1:1)

What this diagram shows:

- Two entities: `employee` and `office`
- One relationship: `has` (connecting employee to office)
- Arrows on BOTH sides mean "at most one"

Reading the diagram:

- Each employee can have **at most one** office
- Each office can be assigned to **at most one** employee
- Some employees might not have an office (partial participation)
- Some offices might be unoccupied (partial participation)

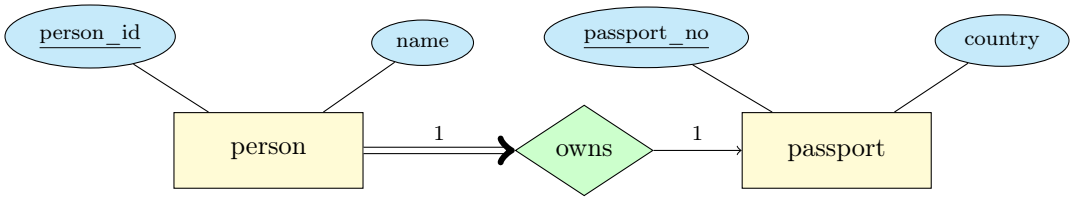
Concrete example with data:

- Employee "John Smith" (ID=E001) has Office 301
- Employee "Jane Doe" (ID=E002) has Office 302
- Employee "Bob Jones" (ID=E003) has NO office (allowed because partial)
- Office 303 is UNOCCUPIED (allowed because partial)

What is NOT allowed:

- John Smith cannot have BOTH Office 301 AND Office 302
- Office 301 cannot be assigned to BOTH John AND Jane

ER Diagram Example: Person-Passport (1:1 Total on Person)



Plain English Explanation: Person-Passport (1:1 Total)

What this diagram shows:

- The **double line** from person means: EVERY person MUST have a passport
- The **single line** from passport means: passports MAY be unissued
- Arrows on both sides mean: maximum one on each side

Reading the diagram:

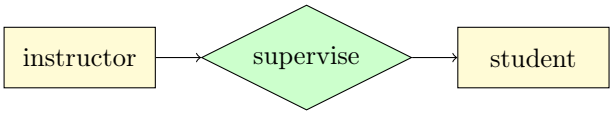
- Each person **must have exactly one** passport (total + 1:1 = exactly one)
- Each passport belongs to **at most one** person

- No person can exist without a passport
- Some passports might not be issued yet (partial on passport side)

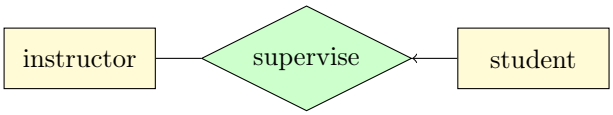
Concrete example:

- Alice (ID=P001) **must have** Passport AB123456
- Bob (ID=P002) **must have** Passport CD789012
- Passport XY999999 exists but not yet issued (allowed - partial on passport)
- Charlie (ID=P003) with NO passport is **NOT allowed** (total participation)

1:1 (arrows both ways)



1:N (arrow to "one" side)



N:1 (arrow from "many")

M:N (no arrows)

One-to-One Examples

Instructor supervises Student:

- Each instructor supervises at most one student
- Each student is supervised by at most one instructor
- Notation: instructor ↔ student (arrows on both ends)

Other examples:

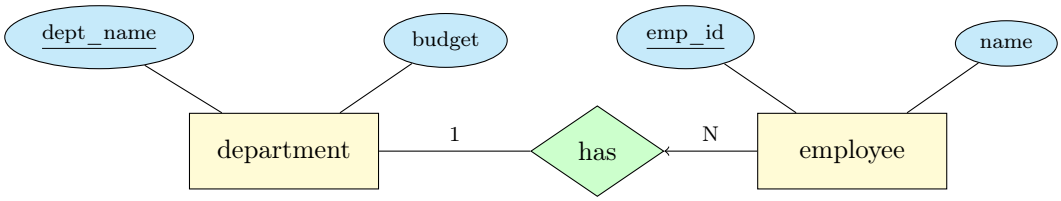
- Employee ↔ Office (one employee per office, one office per employee)
- Country ↔ Capital (one capital per country)
- Person ↔ Passport (in most countries)

4.2 One-to-Many (1:N)

One-to-Many Relationship

Each entity in A can be associated with **many** entities in B.
Each entity in B is associated with **at most one** entity in A.
Graphical Notation: Arrow pointing to the "one" side
Chen Notation: 1 on the "one" side, N on the "many" side
[min, max] Notation: [0, *] on the "many" side, [0, 1] or [1, 1] on the "one" side

ER Diagram Example: Department-Employee (1:N Relationship)



Plain English Explanation: Department-Employee (1:N)

What this diagram shows:

- Arrow points to department = department is the "ONE" side
- No arrow from employee = employee is the "MANY" side
- Label: N on employee side, 1 on department side

Reading the diagram:

- One department can have **many employees** (0, 1, 2, 3, ... employees)
- Each employee works in **at most one department**
- Some departments might have zero employees (partial)
- Some employees might not be assigned to any department (partial)

Concrete example with data:
Computer Science Department:

- Has employees: Alice (E001), Bob (E002), Carol (E003) ← 3 employees

Mathematics Department:

- Has employees: David (E004) ← 1 employee

Physics Department:

- Has NO employees ← 0 employees (allowed - partial)

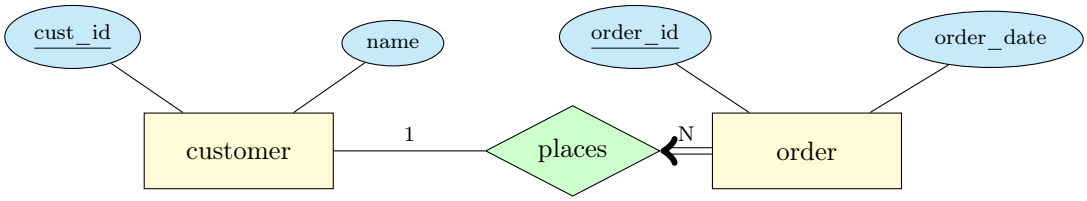
Unassigned:

- Employee Eve (E005) works in NO department (allowed - partial)

What is NOT allowed:

- Alice (E001) cannot work in BOTH CS AND Math departments
- An employee can only be in ONE department (or none)

ER Diagram Example: Customer-Order (1:N with Total on Order)



Plain English Explanation: Customer-Order (1:N Total)

What this diagram shows:

- **Double line** from order: EVERY order MUST belong to a customer
- Arrow to customer: customer is the "ONE" side
- Single line from customer: customers MAY have zero orders

Reading the diagram:

- One customer can place **many orders** (0, 1, 2, ...)
- Each order **must belong to exactly one** customer (total + 1:N)
- Some customers might have zero orders (partial on customer)
- NO order can exist without a customer (total on order)

Concrete example:
Customer Alice (C001):

- Has orders: Order#1001, Order#1002, Order#1003 ← 3 orders

Customer Bob (C002):

- Has NO orders yet ← 0 orders (allowed - partial on customer)

Orders:

- Order#1001 belongs to Alice
- Order#1002 belongs to Alice
- Order#1003 belongs to Alice
- Order#1004 with NO customer is **NOT allowed** (total participation)

One-to-Many Examples

Instructor supervises Student:

- Each instructor can supervise many students (0 or more)
- Each student is supervised by at most one instructor
- Notation: instructor —supervise→ student

Department has Students:

- Each department can have many students
- Each student belongs to at most one department
- Notation: department —belongs— student (arrow to department)

Other examples:

- Department ↔ Employees (one department, many employees)
- Customer ↔ Orders (one customer, many orders)
- Author ↔ Books (one author, many books – simplified)

4.3 Many-to-One (N:1)

Many-to-One Relationship

Many entities in A can be associated with **one** entity in B.
Each entity in A is associated with **at most one** entity in B.
Note: Many-to-one is the inverse of one-to-many.
Graphical Notation: Arrow pointing from the "many" side to the "one" side
Chen Notation: N on the "many" side, 1 on the "one" side
[min, max] Notation: [0, 1] or [1, 1] on the "many" side, [0, *] on the "one" side

Many-to-One Examples

Students belong to Department:

- Many students can belong to one department
- Each student belongs to at most one department
- Notation: student → department (arrow to department)

Other examples:

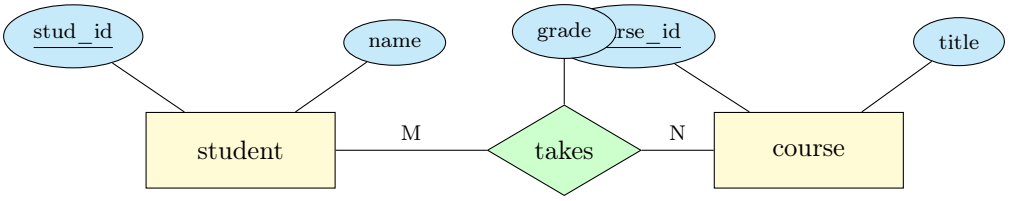
- Employees ↔ Department (many employees, one department)
- Orders ↔ Customer (many orders, one customer)
- Courses ↔ Department (many courses, one department)

4.4 Many-to-Many (M:N)

Many-to-Many Relationship

Entities in A can be associated with **many** entities in B.
Entities in B can be associated with **many** entities in A.
Graphical Notation: Lines without arrows on both sides
Chen Notation: M on one side, N on the other (or N on both)
[min, max] Notation: [0, *] on both sides (or other ranges)

ER Diagram Example: Student-Course (M:N Relationship)



Plain English Explanation: Student-Course (M:N)

What this diagram shows:

- **NO arrows** on either side = many-to-many
- Labels: M on student side, N on course side
- `grade` is an attribute of the RELATIONSHIP (not student or course)

Reading the diagram:

- One student can take **many courses** (0, 1, 2, 3, ...)
- One course can have **many students** (0, 1, 2, 3, ...)
- The `grade` belongs to the (student, course) pair, not to student or course alone

Concrete example with data:
Student Alice (S001):

- Takes: CS101 (grade=A), MATH201 (grade=B), PHYS101 (grade=A)
- ← Alice takes 3 courses

Student Bob (S002):

- Takes: CS101 (grade=C), CS202 (grade=B)
- ← Bob takes 2 courses

Student Carol (S003):

- Takes: NO courses ← 0 courses (allowed - partial)

Course CS101:

- Has students: Alice (grade=A), Bob (grade=C), David (grade=B)
- ← CS101 has 3 students

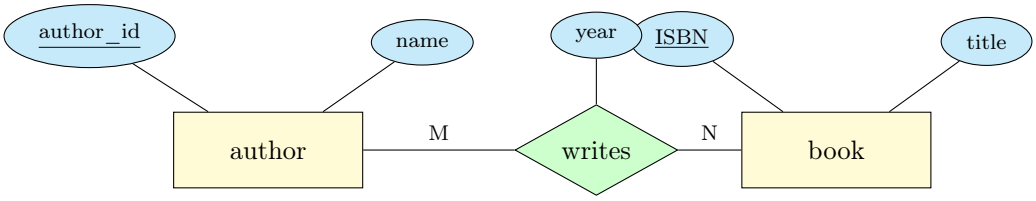
Course HIST301:

- Has NO students ← 0 students (allowed - partial)

Key insight:

- Alice and Bob BOTH take CS101 (allowed in M:N)
- Alice takes MULTIPLE courses (allowed in M:N)
- The grade is different for each (student, course) pair

ER Diagram Example: Author-Book (M:N Relationship)



Plain English Explanation: Author-Book (M:N)

What this diagram shows:

- Many-to-many: NO arrows on either side
- `year` attribute: when author wrote that specific book

Reading the diagram:

- One author can write **many books**
- One book can be written by **many authors** (co-authors)
- The year represents when a specific author contributed to a specific book

Concrete example:
Author: J.K. Rowling (A001):

- Wrote: "Harry Potter 1" (1997), "Harry Potter 2" (1998), "Harry Potter 3" (1999)
- ← 1 author writes MULTIPLE books

Book: "Design Patterns" (ISBN: 978-0201633610):

- Written by: Gamma (1994), Helm (1994), Johnson (1994), Vlissides (1994)
- ← 1 book written by MULTIPLE authors (Gang of Four)

Book: "The C Programming Language" (ISBN: 978-0131103627):

- Written by: Kernighan (1978), Ritchie (1978)
- ← Another example of multiple authors for one book

Why M:N is needed:

- One author (Rowling) writes many books
- One book (Design Patterns) has many authors
- Cannot model this with 1:N or N:1 without losing information

Step-by-Step: Determining Cardinality from Requirements

Step	Question to Ask	Example Requirement	Answer	Result
1	Can one entity in A be related to multiple entities in B?	"Can one instructor teach multiple courses?"	Yes	A side: many
2	Can one entity in B be related to multiple entities in A?	"Can one course be taught by multiple instructors?"	No	B side: one
3	Combine results from steps 1 & 2	Instructor-Course relationship	Many-One	1:N
1	Can one student take multiple courses?	"Students may enroll in several courses"	Yes	Student: many
2	Can one course have multiple students?	"Courses have many enrolled students"	Yes	Course: many
3	Combine	Student-Course relationship	Many-Many	M:N
1	Can one employee have multiple offices?	"Each employee has one office"	No	Employee: one
2	Can one office have multiple employees?	"Offices are assigned to one employee"	No	Office: one
3	Combine	Employee-Office relationship	One-One	1:1

Many-to-Many Examples

Instructor supervises Student:

- Each instructor can supervise many students
- Each student can be supervised by many instructors
- Notation: instructor —supervise— student (no arrows)

Student takes Course:

- Each student can take many courses
- Each course can be taken by many students
- Notation: student —takes— course

Other examples:

- Authors ↔ Books (one book can have multiple authors, one author can write multiple books)
- Students ↔ Clubs (students can join multiple clubs, clubs have multiple students)
- Actors ↔ Movies (one actor can be in multiple movies, one movie has multiple actors)

4.5 Notation Comparison

Three Cardinality Notations

All three notations are equivalent. Choose based on preference or convention.

Graphical Notation (Arrow):

- Arrow (→) = "one"
- Line (—) = "many"
- Clear and visual

Chen Notation (1, N, M):

- 1 = one
- N, M = many
- Traditional and widely used

[min, max] Notation:

- Most expressive

0, 1 = at most one (partial)

1, 1 = exactly one (total)

0, * = zero or more (partial, many)

1, * = one or more (total, many)

- Can express nuanced constraints

Visual Understanding: Cardinality as Set Mappings

Type	Visual Mapping	Constraint Description
1:1	Set A: {a1, a2, a3} → Set B: {b1, b2, b3} a1 → b1, a2 → b2, a3 → (none)	Each element in A maps to at most one element in B. Each element in B is mapped by at most one element in A.
1:N	Set A: {a1, a2} → Set B: {b1, b2, b3, b4} a1 → {b1, b2}, a2 → {b3, b4}	Each element in A can map to multiple elements in B. Each element in B is mapped by at most one element in A.
N:1	Set A: {a1, a2, a3} → Set B: {b1, b2} a1 → b1, a2 → b1, a3 → b2	Multiple elements in A can map to the same element in B. Each element in A maps to at most one element in B.
M:N	Set A: {a1, a2} → Set B: {b1, b2, b3} a1 → {b1, b2}, a2 → {b2, b3}	Each element in A can map to multiple elements in B. Each element in B can be mapped by multiple elements in A.

Comprehensive Cardinality Notation Comparison Table

Cardinality	Graphical	Chen	[min,max]	Example
1:1 (both partial)	A ↔ B (arrows both ways)	A (1)—(1) B	A [0,1]—[0,1] B	instructor supervises student (each at most one)
1:1 (total on B)	A ↔ B (double line from B)	A (1)===(1) B (double line from B)	A [0,1]—[1,1] B	employee has office (every employee must have office)
1:N (both partial)	A —→ B (arrow to B)	A (N)—(1) B	A [0,1]—[0,*] B	student belongs to department (student in at most 1 dept)
1:N (total on A)	A ===→ B (double line from A)	A (N)===(1) B (double line from A)	A [1,1]—[0,*] B	student belongs to department (every student has department)
M:N (both partial)	A — B (no arrows)	A (M)—(N) B	A [0,*]—[0,*] B	student takes course (both many-to-many)
M:N (total on A)	A ===== B (double line from A)	A (M)===(N) B (double line from A)	A [1,*]—[0,*] B	student takes course (every student takes ≥ 1 course)

Attribute Types Summary Table

Attribute Type	Description	Example	ER Symbol
Simple	Cannot be divided	SSN, age, salary	Simple oval
Composite	Divided into subparts	address (street, city, zip)	Oval with connections
Single-valued	One value per entity	birth_date, ID	Simple oval
Multi-valued	Multiple values per entity	phone_numbers, email	Double oval
Stored	Explicitly stored in DB	birth_date, name	Simple oval
Derived	Computed from other attrs	age (from birth_date)	Dashed oval
Key	Uniquely identifies entity	ID, course_id	Underlined oval
Discriminator	Partial key for weak entity	sec_id, room_number	Dashed underline

Notation Translation Examples

1:1 Relationship (Partial on both sides):

- Graphical: instructor ↔ student (arrows both ways)
- Chen: instructor (1)—supervise—(1) student

min, max : instructor [0,1]—supervise—[0,1] student

1:N Relationship (Partial one side, total many side):

- Graphical: instructor —supervise→ student (arrow to instructor, double line from student)
- Chen: instructor (1)—supervise—(N) student (double line from student)

min, max : instructor [0,*]—supervise—[1,1] student

M:N Relationship (Both partial):

- Graphical: instructor —supervise— student (no arrows)
- Chen: instructor (M)—supervise—(N) student

min, max : instructor [0,*]—supervise—[0,*] student

5 Participation Constraints

Participation

Total Participation: Every entity in the entity set participates in at least one relationship.

- Depicted with **double line** in ER diagrams
- In [min, max]: min ≥ 1

Partial Participation: Some entities may not participate in any relationship.

- Depicted with **single line** in ER diagrams
- In [min, max]: min $= 0$

Participation Examples

Total Participation:

- Every student must be enrolled in at least one course
- student (double line) —takes— course (arrow to student)

min, max : student [1, *]—takes—[0, *] course

Partial Participation:

- Some instructors may not teach any courses
- instructor (single line) —teaches— course

min, max : instructor [0, *]—teaches—[0, *] course

Mixed:

- Every student must belong to exactly one department
- Departments may have zero students
- student [1,1]—belongs—[0,*] department

Common Participation Mistakes

Mistake 1: Confusing cardinality with participation

- Cardinality: How many entities can be related
- Participation: Whether all entities must be related

Mistake 2: Assuming total participation by default

- Always explicitly check requirements
- Example: Not all employees have an assigned office (partial)

Mistake 3: Overusing total participation

- Only use when requirement explicitly states "must" or "every"
- Example: "Students may take courses" → partial, not total

Participation Constraint Decision Table

Requirement Statement	Participation	Notation
"Every student **must** belong to a department"	Total	Double line
"Students **may** take courses"	Partial	Single line
"All employees **are assigned** to a department"	Total	Double line
"Some instructors teach courses"	Partial	Single line
"Each course **has at least one** section"	Total	Double line
"Customers can rent video tapes"	Partial	Single line

Combining Cardinality and Participation Examples

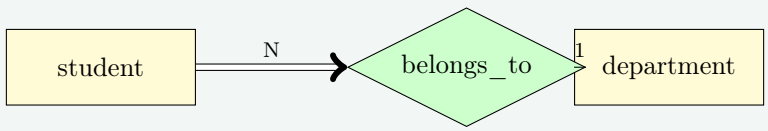
Scenario	Card.	Particip.	[min,max]	Meaning
Student must belong to exactly one department	N:1	Total (student)	[1,1]—[0,*]	Every student in exactly 1 dept
Department may have many students	1:N	Partial (dept)	[0,1]—[0,*]	Department can have 0 or more students
Instructor may supervise many students	1:N	Partial (both)	[0,*]—[0,1]	Instructor: 0+ students, Student: 0-1 instructor
Student must take at least one course	M:N	Total (student)	[1,*]—[0,*]	Every student takes $\geq$ 1 course
Course must have at least one section	1:N	Total (section)	[0,*]—[1,1]	Every section belongs to exactly 1 course
Employee must have exactly one office	1:1	Total (employee)	[1,1]—[0,1]	Every employee has 1 office, office may be empty

5.1 ER Diagrams: Chen vs. [Min,Max] Notation

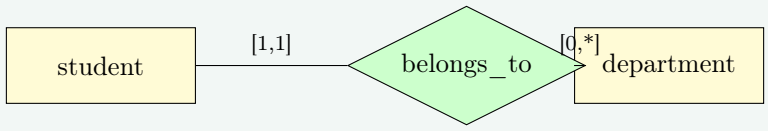
Example 1: Student-Department (N:1

Scenario: Student must belong to exactly one department

Chen Notation:



[Min,Max] Notation:

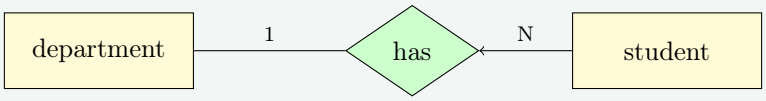


Plain English: Every student MUST belong to exactly 1 department (total participation). Department can have 0 or more students (partial).

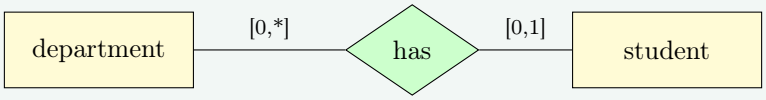
Example 2: Department-Student (1:N

Scenario: Department may have many students

Chen Notation:



[Min,Max] Notation:

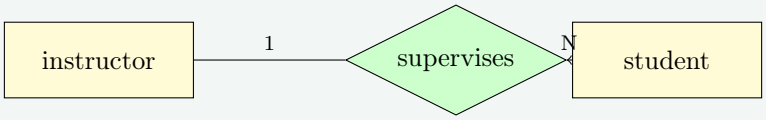


Plain English: Department can have 0 or more students. Student belongs to at most 1 department (both partial).

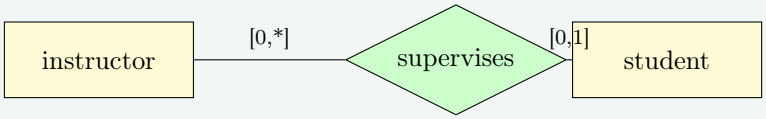
Example 3: Instructor-Student (1:N

Scenario: Instructor may supervise many students

Chen Notation:



[Min,Max] Notation:

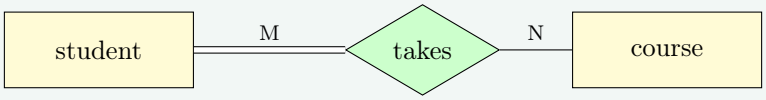


Plain English: Instructor can supervise 0+ students. Student can have 0-1 instructor (both partial participation).

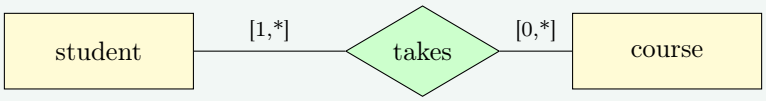
Example 4: Student-Course (M:N

Scenario: Student must take at least one course

Chen Notation:



[Min,Max] Notation:

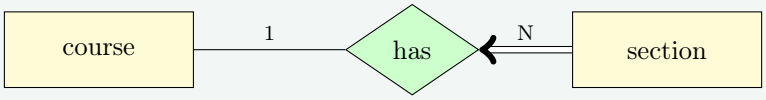


Plain English: Every student MUST take at least 1 course (total: [1,*]). Course can have 0+ students (partial: [0,*]).

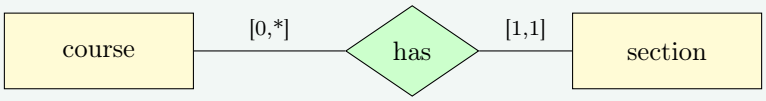
Example 5: Course-Section (1:N

Scenario: Course must have at least one section (actually section must belong to course)

Chen Notation:



[Min,Max] Notation:

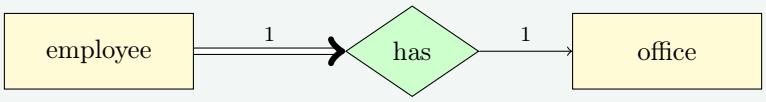


Plain English: Every section MUST belong to exactly 1 course (total: [1,1]). Course can have 0+ sections (partial: [0,*]).

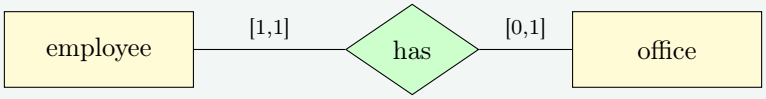
Example 6: Employee-Office (1:1

Scenario: Employee must have exactly one office

Chen Notation:



[Min,Max] Notation:



Plain English: Every employee MUST have exactly 1 office (total: [1,1]). Office can be assigned to 0-1 employee (partial: [0,1]).

6 Keys

6.1 Primary Keys for Entity Sets

Primary Key for Entity Sets

Key: A set of attributes that uniquely identify entities in an entity set.
Properties:

- **Uniqueness**: No two entities have the same key value
- **Minimality**: No proper subset is also a key

Primary Key: The key chosen to identify entities.

- Underlined in ER diagrams
- Cannot be NULL

Entity Primary Key Examples

Simple (single-attribute) keys:

- instructor: ID
- course: course_id
- student: ID
- department: dept_name

Composite (multi-attribute) keys:

- section: (course_id, sec_id, semester, year)
- time_slot: (time_slot_id, day, start_time)

6.2 Primary Keys for Relationship Sets

Primary Key for Relationship Sets

To distinguish relationships in a relationship set, we use:
Base components: Union of primary keys from participating entity sets.
Additional components: Attributes of the relationship itself (if any).
General rule: Primary key = (key_1, key_2, ..., key_n, attr_1, ..., attr_m)
Where key_i are entity primary keys and attr_j are relationship attributes.

Cardinality Affects Relationship Keys

The mapping cardinality determines which subset of attributes forms the primary key:
Many-to-Many (M:N):

- Use union of all participating entity keys
- Example: supervise(instructor.ID, student.ID)

One-to-Many (1:N):

- Use primary key of the "many" side
- Example: supervise(student.ID) – student is on "many" side

Many-to-One (N:1):

- Use primary key of the "many" side
- Example: belongs(student.ID) – student is on "many" side

One-to-One (1:1):

- Use primary key of either participating entity
- Example: supervise(instructor.ID) OR supervise(student.ID)

Key Selection Rules Based on Cardinality

Cardinality	Relationship Primary Key	Explanation
M:N	(key_A, key_B)	Need both keys to uniquely identify relationship instance
1:N	key_A (many side)	Each A entity participates at most once, so key_A uniquely identifies the relationship
N:1	key_A (many side)	Each A entity participates at most once, so key_A uniquely identifies the relationship
1:1	key_A OR key_B	Either key works since each entity participates at most once
Note: If relationship has additional attributes, include them only if needed for uniqueness		

Key Selection in Practice

Example 1: Student takes Course (M:N)

- Relationship: takes(student, course)
- Additional attributes: semester, year, grade
- Cardinality: student [0,*]—takes—[0,*] course
- **Primary key**: (student.ID, course.course_id, semester, year)
- Why? A student can take the same course in different semesters

Example 2: Student belongs to Department (N:1)

- Relationship: belongs(student, department)
- Cardinality: student [1,1]—belongs—[0,*] department
- **Primary key**: (student.ID)
- Why? Each student belongs to exactly one department, so student.ID uniquely identifies the relationship

Example 3: Instructor supervises Student (1:1)

- Relationship: supervise(instructor, student)
- Cardinality: instructor [0,1]—supervise—[0,1] student
- **Primary key**: (instructor.ID) OR (student.ID)
- Why? Each instructor supervises at most one student, and vice versa

Relationship Primary Key Examples

M:N relationship:

- takes(student, course)
- Primary key: (student.ID, course.course_id, sec_id, semester, year)

- All entity keys needed to uniquely identify the relationship

1:N relationship:

- belongs(student, department)
- Cardinality: student [1,1]—belongs—[0,*] department
- Primary key: (student.ID) – "many" side is student
- Each student belongs to one department, so student.ID uniquely identifies the relationship

1:1 relationship:

- supervise(instructor, student)
- Cardinality: instructor [0,1]—supervise—[0,1] student
- Primary key: (instructor.ID) OR (student.ID) – either works

7 Weak Entity Types

Weak Entity Type

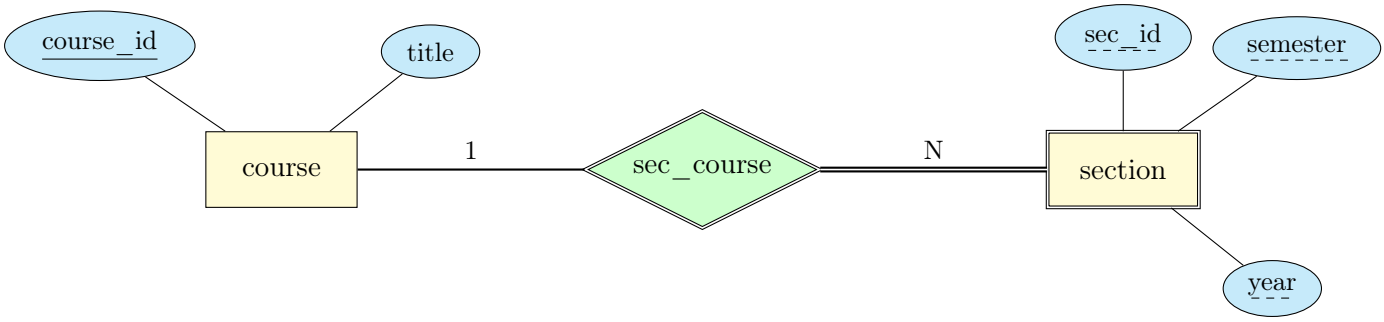
Weak Entity: An entity type whose existence depends on another entity.
Characteristics:

- **No primary key** of its own
- **Identifying entity**: The strong entity it depends on
- **Identifying relationship**: The relationship connecting them
- **Discriminator (partial key)**: Attributes that distinguish entities within the same identifying entity

Notation:

- Weak entity: **double rectangle**
- Identifying relationship: **double diamond**
- Discriminator: **dashed underline**

Primary key of weak entity: (identifying entity key) + (discriminator)



Section depends on Course
Primary key: (course_id, sec_id, semester, year)

Plain English Explanation: Course-Section Weak Entity

What this diagram shows:

- **Double rectangle** around section = weak entity
- **Double diamond** for sec_course = identifying relationship
- **Dashed underline** on sec_id, semester, year = discriminators (partial key)
- **Single rectangle** around course = strong entity

Why is section weak?

- Section **cannot exist** without a course
- sec_id alone is NOT unique (many courses can have "Section 1")
- Must know WHICH course to identify a specific section

Concrete example with data:
Course: CS101 (Intro to Programming)

- Section 1, Fall 2024 → Full ID: (CS101, 1, Fall, 2024)
- Section 2, Fall 2024 → Full ID: (CS101, 2, Fall, 2024)
- Section 1, Spring 2025 → Full ID: (CS101, 1, Spring, 2025)

Course: MATH201 (Calculus)

- Section 1, Fall 2024 → Full ID: (MATH201, 1, Fall, 2024)
- ← Notice: "Section 1" exists for BOTH CS101 and MATH201

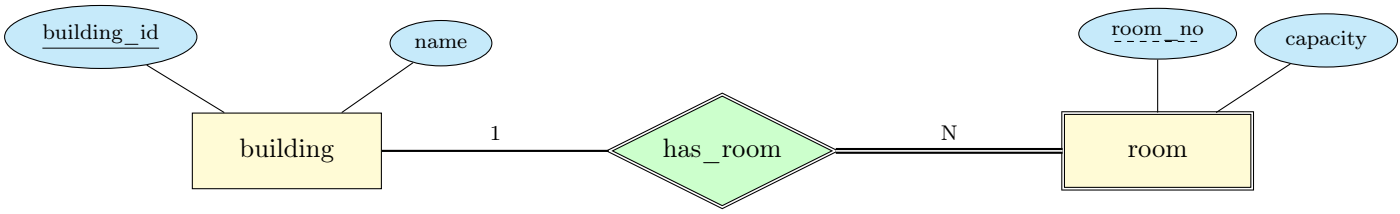
Key insight:

- "Section 1" alone is ambiguous (which course?)
- Need course_id to uniquely identify: CS101's Section 1 MATH201's Section 1
- Primary key = (course_id, sec_id, semester, year)
- Cannot have Section 1, Fall 2024 without specifying WHICH course

Total participation (double line):

- Every section MUST belong to exactly one course
- No orphan sections allowed
- If a course is deleted, all its sections must be deleted too

ER Diagram Example: Building-Room (Another Weak Entity)



Plain English Explanation: Building-Room Weak Entity

What this diagram shows:

- room is a **weak entity** (double rectangle)
- room depends on building
- room_no is the **discriminator** (dashed underline)

Why is room weak?

- "Room 101" exists in MANY buildings
- Room 101 in Building A Room 101 in Building B
- Must know WHICH building to identify a specific room

Concrete example:
Engineering Building (B001):

- Room 101 → Full ID: (B001, 101), Capacity: 50
- Room 102 → Full ID: (B001, 102), Capacity: 30
- Room 201 → Full ID: (B001, 201), Capacity: 40

Science Building (B002):

- Room 101 → Full ID: (B002, 101), Capacity: 60 ← Same room_no, different building!
- Room 102 → Full ID: (B002, 102), Capacity: 35

Primary key of room:

- (building_id, room_no)
- Building B001's Room 101 Building B002's Room 101
- "Room 101" alone is NOT unique across the system

What happens if we delete a building?

- If Engineering Building (B001) is deleted...
- ALL its rooms (101, 102, 201) must also be deleted
- Because rooms cannot exist without a building (total participation)

Weak Entity Example: Course Sections - Original

Scenario:

- course: strong entity with key course_id
- section: weak entity depending on course
- Discriminator: sec_id, semester, year
- Relationship: sec_course (identifying relationship)

Primary key of section:

- (course_id, sec_id, semester, year)
- course_id comes from the identifying entity
- sec_id, semester, year are the discriminator

Why weak?

- A section cannot exist without a course
- sec_id alone doesn't uniquely identify a section (CS-101 section 1, BIO-101 section 1)
- Must know which course to uniquely identify the section

Weak Entity Constraints

Required conditions:

1. Weak entity must have **total participation** in identifying relationship
2. Identifying relationship must be **1:N** or **1:1** (strong entity on "1" side)
3. Cannot have **M:N** identifying relationships

Why total participation?

- Weak entity cannot exist without identifying entity
- Every weak entity instance must be related to a strong entity instance

Why 1:N or 1:1?

- Weak entity must be uniquely associated with one strong entity
- M:N would make the identifying entity ambiguous

More Weak Entity Examples

Building and Room:

- building: strong entity, key = building_id
- room: weak entity, discriminator = room_number
- Primary key of room: (building_id, room_number)

- Room 101 in Building A $\neq$ Room 101 in Building B

Order and Order_Item:

- order: strong entity, key = order_id
- order_item: weak entity, discriminator = item_number
- Primary key of order_item: (order_id, item_number)
- Item 1 of Order 100 $\neq$ Item 1 of Order 200

Employee and Dependent:

- employee: strong entity, key = employee_id
- dependent: weak entity, discriminator = dependent_name
- Primary key of dependent: (employee_id, dependent_name)
- Assumes dependent names are unique within one employee's dependents

8 Specialization

Specialization

Specialization: A top-down design process where we designate subgroupings within an entity set.
Generalization: A bottom-up design process where we combine entity sets with common attributes into a higher-level entity set.
ISA Relationship: "is a" relationship between specialized and generalized entities.

- Depicted by a triangle labeled ISA
- Example: instructor "is a" person

Attribute Inheritance: Lower-level entities inherit all attributes and relationships from higher-level entities.

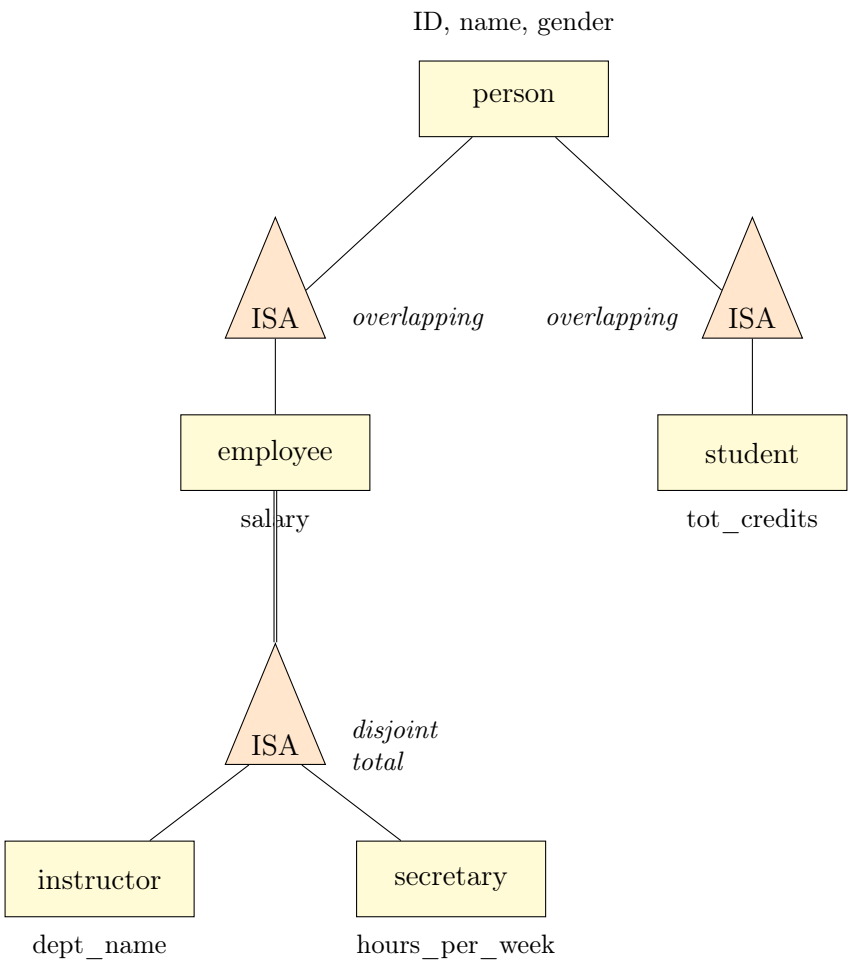
Specialization Example: Person

Person entity hierarchy:

- `person`: ID, name, gender (higher-level entity)
- `employee` ISA `person`: inherits (ID, name, gender) + adds salary
- `student` ISA `person`: inherits (ID, name, gender) + adds tot_credits

Further specialization:

- `instructor` ISA `employee`: inherits all employee attributes + adds dept_name
- `secretary` ISA `employee`: inherits all employee attributes + adds hours_per_week



8.1 Overlapping vs. Disjoint Specialization

Overlapping vs. Disjoint

Disjoint Specialization:

- An entity can belong to **at most one** specialized entity set
- Depicted with a **single ISA triangle** with arrows pointing to it
- Example: instructor and secretary are disjoint (employee is one or the other, not both)

Overlapping Specialization:

- An entity can belong to **multiple** specialized entity sets
- Depicted with **separate ISA triangles**
- Example: employee and student are overlapping (a person can be both)

Overlapping vs. Disjoint Examples

Overlapping:

- person → employee, student

- A teaching assistant is both an employee and a student
- Uses separate ISA symbols

Disjoint:

- employee → instructor, secretary
- An employee is either an instructor or a secretary, not both
- Uses single ISA symbol with multiple arrows

Vehicle hierarchy (disjoint):

- vehicle → car, truck, motorcycle
- A vehicle is exactly one type

Account hierarchy (overlapping):

- account → savings_account, checking_account
- Some accounts can be both savings and checking

8.2 Total vs. Partial Specialization

Total vs. Partial Specialization

Total Specialization:

- Every entity in the higher-level entity set must belong to at least one lower-level entity set
- Depicted with **double line** from entity to ISA triangle
- Example: Every employee must be either instructor or secretary

Partial Specialization:

- Some entities may not belong to any lower-level entity set
- Depicted with **single line** from entity to ISA triangle
- Example: Some persons are neither employees nor students

Total vs. Partial Examples

Partial:

- person → employee, student
- Not every person is an employee or student (e.g., retirees, children)
- Single line from person to ISA

Total:

- employee → instructor, secretary

- Every employee must be classified as instructor or secretary
- Double line from employee to ISA

Account (total, disjoint):

- account → savings, checking
- Every account must be either savings or checking
- Double line, single ISA triangle

9 Practice Exercises with Solutions

9.1 Exercise 1: Banking System

Banking System Requirements

Requirements:

- Banks have branches located in different cities
- Customers can have multiple accounts at different branches
- Each account belongs to exactly one branch
- Customers can have multiple loans from different branches
- Each loan is from exactly one branch
- Track: bank(bank_name, headquarters), branch(branch_name, city, assets), customer(customer_id, name, address), account(account_number, balance), loan(loan_number, amount)

Solution - Key Design Decisions:

1. **Entities**: bank, branch, customer, account, loan
2. **Weak entities**: branch (depends on bank, discriminator: branch_name)
3. **Relationships**:
   - bank_branch: bank [1,1] — [0,*] branch (identifying relationship)
   - customer_account: customer [1,*] — [0,*] account (M:N, "has")
   - branch_account: branch [1,1] — [0,*] account (1:N)
   - customer_loan: customer [1,*] — [0,*] loan (M:N, "borrows")
   - branch_loan: branch [1,1] — [0,*] loan (1:N)
4. **Primary Keys**:
   - bank: (bank_name)
   - branch: (bank_name, branch_name) – composite from bank + discriminator
   - customer: (customer_id)
   - account: (account_number)
   - loan: (loan_number)

9.2 Exercise 2: Hospital Management

Hospital Requirements and Solution

Requirements:

- Hospitals have multiple departments (Cardiology, Neurology, etc.)
- Doctors work in exactly one department
- Patients can be treated by multiple doctors
- Each treatment has a date and diagnosis
- Departments have beds, each bed has a bed_number
- Patients can be assigned to at most one bed at a time

Solution:

- **Entities**: hospital, department, doctor, patient, bed
- **Weak entity**: bed (depends on department, discriminator: bed_number)
- **Relationships**:
  - has_dept: hospital [1,1] — [0,*] department (1:N)
  - works_in: doctor [1,1] — [0,*] department (N:1, total on doctor)
  - treats: doctor [0,*] — [0,*] patient (M:N, attributes: date, diagnosis)
  - dept_bed: department [1,1] — [0,*] bed (1:N, identifying)
  - assigned_to: patient [0,1] — [0,*] bed (1:N, partial on patient)
- **Cardinality rationale**:
  - Doctor works in exactly one department: N:1 with total participation
  - Patient can be treated by many doctors, doctor can treat many patients: M:N
  - Patient can be in at most one bed, but bed can be unoccupied: 1:N

9.3 Exercise 3: Course Prerequisites (Recursive Relationship)

Course Prerequisites

Requirement: "Courses can have prerequisite courses. For example, Data Structures is a prerequisite for Algorithms."
Solution:

- **Entity**: course(course_id, title, credits)
- **Recursive Relationship**: prereq
- **Roles**:
  - course_id: The course that has a prerequisite
  - prereq_id: The course that is the prerequisite
- **Cardinality**: M:N
  - A course can have multiple prerequisites
  - A course can be a prerequisite for multiple courses
- **Primary Key**: (course_id, prereq_id)

Example instances:

- (Algorithms, Data Structures) – Algorithms requires Data Structures
- (Algorithms, Discrete Math) – Algorithms requires Discrete Math
- (Databases, Data Structures) – Databases requires Data Structures

9.4 Exercise 4: Employee Hierarchy (Specialization)

Employee Specialization

Requirement: "Employees can be managers, engineers, or salespeople. Managers have a bonus attribute. Engineers have a specialty (software, hardware, etc.). Salespeople have a commission_rate. An employee must be exactly one of these types."
Solution:

- **Superclass**: employee(employee_id, name, salary)
- **Subclasses**:
  - manager(bonus) ISA employee
  - engineer(specialty) ISA employee
  - salesperson(commission_rate) ISA employee
- **Specialization type**: Disjoint, Total
  - **Disjoint**: An employee is exactly one type (manager, engineer, or salesperson), not multiple
  - **Total**: Every employee must be classified as one of these types
- **Notation**: Single ISA triangle with double line from employee

9.5 Exercise 5: University Person Hierarchy

Overlapping Specialization Example

Requirement: "The university has people who can be students, employees, or both. Students have tot_credits. Employees have salary. Some people (teaching assistants) are both students and employees."
Solution:

- **Superclass**: person(ID, name, address)
- **Subclasses**:
  - student(tot_credits) ISA person
  - employee(salary) ISA person
- **Specialization type**: Overlapping, Partial
  - **Overlapping**: A person can be both student and employee (e.g., teaching assistant)
  - **Partial**: Some people are neither students nor employees (e.g., visitors, retirees)
- **Notation**: Separate ISA triangles, single lines from person

Example instances:

- Zhang: student only (ID=00128, tot_credits=100)
- Einstein: employee only (ID=10101, salary=95000)

- Tanaka: both student and employee (ID=55555, tot_credits=30, salary=20000) – teaching assistant

10 Complete ER Diagram Examples from Course

10.1 Video Store Database

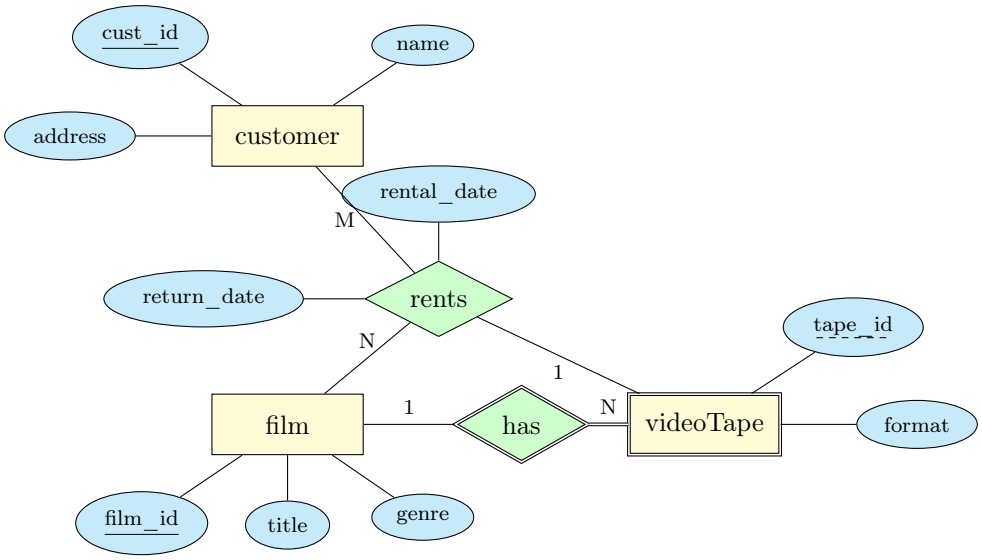
Video Store Requirements

A video store needs to track:

- **Customers**: ID, name, address
- **Films**: film_id, title, genre, rating
- **Video Tapes**: tape_id, format (VHS, DVD), purchase_date
- Each film can have multiple video tape copies
- Customers rent video tapes (not films directly)
- Track rental: rental_date, return_date

Key observations:

- Film and VideoTape are separate entities (one film, many tapes)
- Weak entity: VideoTape depends on Film
- Ternary relationship: customer rents videoTape of film



VideoTape is a weak entity depending on Film.
Rental is a ternary relationship:
customer rents videoTape of film.

Video Store Design Decisions

Why Film and VideoTape are separate:

- Multiple copies of same film (different tape_ids)
- Each tape has its own rental history
- Tapes can have different formats (VHS, DVD)

Why VideoTape is weak:

- tape_id alone doesn't uniquely identify a tape
- Need (film_id, tape_id) to uniquely identify
- A tape cannot exist without a film

Why ternary relationship:

- Customer rents a specific tape, not just any tape of a film
- Need to know: which customer, which tape, which film
- Cannot decompose into binary without losing information

10.2 Complete University Database ER Diagram

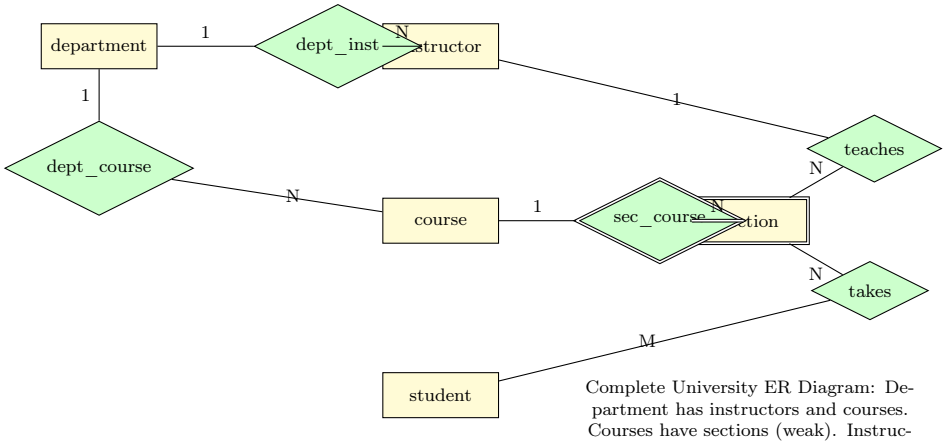
University Database Entities and Relationships

Entities:

- instructor(ID, name, dept_name, salary)
- student(ID, name, dept_name, tot_cred)
- course(course_id, title, dept_name, credits)
- section(sec_id, semester, year, building, room, time_slot) – weak, depends on course
- department(dept_name, building, budget)

Relationships:

- teaches: instructor ↔ section (1:N)
- takes: student ↔ section (M:N), attributes: grade
- sec_course: course ↔ section (1:N, identifying)
- dept_inst: department ↔ instructor (1:N)
- dept_course: department ↔ course (1:N)



Complete University ER Diagram: Department has instructors and courses. Courses have sections (weak). Instructors teach sections. Students take sections.

10.3 Library Database Exercise

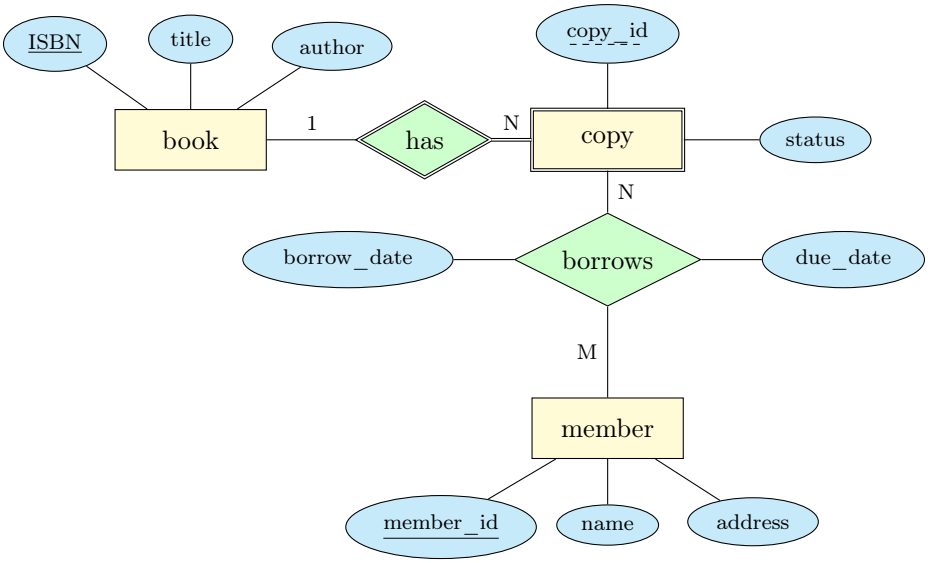
Library Database Requirements

Design an ER diagram for a library with the following requirements:

- Track **books**: ISBN (key), title, author, publisher, year
- Track **members**: member_id (key), name, address, phone
- Track **copies**: copy_id (discriminator), status (available/borrowed)
- Each book can have multiple physical copies
- Members can borrow copies (not books directly)
- Track borrowing: borrow_date, due_date, return_date

Solution approach:

- book: strong entity with ISBN as key
- copy: weak entity depending on book (discriminator: copy_id)
- member: strong entity with member_id as key
- borrows: relationship between member and copy (M:N)



Library Database: Copy is
weak entity depending on Book.
Members borrow copies (M:N relationship).

11 Design Best Practices

Decision Flowchart: When to Use Weak Entities

Question	If YES	If NO	Example
Does the entity have a natural primary key?	Use strong entity	Continue to next question	course has course_id → strong
Does the entity's existence depend on another entity?	Continue to next question	Use strong entity	section depends on course
Can the entity exist without the other entity?	Use strong entity	Continue to next question	student can exist alone
Does the entity have a partial key (discriminator)?	Use weak entity	Reconsider design	sec_id is partial key
Is there a meaningful identifying relationship?	Use weak entity	Use strong entity	sec_course identifies section

Design Decision Matrix: Entity vs. Attribute vs. Relationship

Concept	Model as Entity if...	Model as Attribute if...	Example
Phone	Has multiple properties (type, area code, number) or participates in relationships	Simple single value	phone entity with (type, number) vs. phone attribute
Address	Need to query by parts (city, zip) or shared by multiple entities	Simple string, not queried separately	address(street, city, zip) vs. "123 Main St"
Date	Need to track multiple dates or participate in relationships	Single date value	birth_date attribute vs. appointment entity
Enrollment	Has attributes (grade, date) or participates in other relationships	Simple connection	takes relationship with grade attribute

Specialization Design Decisions

Requirement	Overlapping?	Total?	Rationale
"Every employee is either a manager, engineer, or salesperson"	No (Disjoint)	Yes	Cannot be multiple types simultaneously; all must be classified
"People can be students, employees, or both"	Yes	No	Teaching assistants are both; some people are neither
"All accounts are either savings or checking"	No (Disjoint)	Yes	Account must be one type; all accounts classified
"Vehicles can be cars, trucks, or motorcycles"	No (Disjoint)	Yes	A vehicle is exactly one type
"Customers can be individuals or companies, and some are both"	Yes	No	Corporate customers with personal accounts; not all entities classified

ER Modeling Heuristics

Entity vs. Attribute:

- Use entity if it has attributes of its own or participates in relationships
- Use attribute if it's a simple property
- Example: "phone" as attribute vs. "phone" entity with attributes (type, number, area_code)

Entity vs. Relationship:

- Use relationship for associations between entities
- Use entity if the association has significant attributes or participates in other relationships
- Example: "enrollment" could be a relationship (takes) or an entity (enrollment entity with grade, semester, etc.)

Binary vs. Ternary:

- Prefer binary relationships when possible (easier to understand and implement)
- Use ternary only when binary decomposition loses information

Redundancy:

- Avoid redundant relationships
- Example: If A $\rightarrow$ B and B $\rightarrow$ C, don't add A $\rightarrow$ C unless necessary

Comprehensive ER Modeling Troubleshooting Guide

Problem/Question	Analysis	Solution
"Should 'email' be an entity or attribute?"	Does email have properties? Do you need to query email separately?	If simple (single value): attribute. If complex (type, verified status, primary): entity
"Is section a weak entity?"	Can section exist without course? Does it have natural key?	Yes, weak. Section only meaningful with course. Key: (course_id, sec_id, semester, year)
"What cardinality: student-course?"	Can student take multiple courses? Can course have multiple students?	Both yes → M:N relationship
"Should 'enrollment' be a relationship or entity?"	Does enrollment participate in other relationships? Has significant attributes?	If just connecting student-course with grade: relationship. If has payment, status, etc.: entity
"Partial or total participation for student in 'takes'?"	Must every student take at least one course?	If "must": total (double line). If "can": partial (single line)
"How to model multi-valued attribute 'phone_numbers'?"	Need to store multiple phones per entity	Option 1: Multi-valued attribute (double oval). Option 2: Separate phone entity (better for queries)
"Is this specialization overlapping or disjoint?"	Can an entity be in multiple subclasses simultaneously?	If yes (e.g., student & employee): overlapping. If no (e.g., savings vs. checking): disjoint
"What's the primary key for M:N relationship?"	Need to uniquely identify each relationship instance	Union of primary keys from both entities, plus any relationship attributes needed for uniqueness
"Should I use ternary or binary relationships?"	Can the relationship be decomposed without losing information?	Prefer binary. Use ternary only if decomposition loses information (e.g., proj_guide)

Real-World Modeling Scenarios with Solutions

Scenario	Key Design Decisions	ER Components
Amazon Product System	Products have multiple sellers; customers have wishlists; orders contain items	Entities: customer, product, seller, order. Weak: order_item. M:N: customer-product (wishlist), seller-product. 1:N: order-order_item
Airline Reservation	Flights have instances (flight_number on specific date); passengers book seats	Entities: flight, flight_instance, passenger, seat. Weak: seat, flight_instance. M:N: passenger-flight_instance (booking)
Social Network	Users friend each other (symmetric); users post messages; posts have comments	Entities: user, post, comment. Recursive: friendship (user-user). 1:N: user-post, post-comment
Hotel Management	Hotels have rooms; rooms have types; guests make reservations	Entities: hotel, room_type, room, guest, reservation. Weak: room. M:N: guest-room (via reservation with dates)
Online Course Platform	Courses have modules; modules have lessons; students enroll and track progress	Entities: course, module, lesson, student. Weak: module (on course), lesson (on module). M:N: student-lesson (progress)

Common ER Modeling Mistakes

1. **Misidentifying entities:**
   - Using verbs as entities (should be relationships)
   - Using attributes as entities or vice versa

2. **Incorrect cardinalities:**
   - Not carefully reading requirements
   - Confusing "can" (partial) with "must" (total)
   - Example: "Students can take courses" $\neq$ "Students must take courses"
3. **Missing weak entities:**
   - Not recognizing existence dependencies
   - Example: Section depends on Course
4. **Overusing ternary relationships:**
   - Using ternary when binary would suffice
   - Makes schema more complex than necessary
5. **Forgetting primary keys:**
   - Every strong entity must have a primary key
   - Weak entities need discriminators
6. **Incorrect specialization:**
   - Using disjoint when should be overlapping
   - Using total when should be partial

12 Summary

Key Takeaways

ER Model Components:

- **Entity Types**: Objects (rectangles)
- **Attributes**: Properties (ovals) – simple/composite, single/multi-valued, stored/derived
- **Relationships**: Associations (diamonds)
- **Roles**: Functions in relationships (especially for recursive)

Cardinality Constraints (for binary):

- 1:1 – one-to-one
- 1:N – one-to-many
- N:1 – many-to-one
- M:N – many-to-many

Participation:

- Total (double line): every entity must participate
- Partial (single line): some entities may not participate

Keys:

- Entity: attributes that uniquely identify (underlined)
- Relationship: depends on cardinality (union of keys or key of "many" side)

Weak Entities:

- Depend on identifying entity (double rectangle)
- Identifying relationship (double diamond)
- Discriminator (dashed underline)
- Must have total participation and 1:N or 1:1 relationship with strong entity

Specialization:

- ISA hierarchy (triangle)
- Attribute inheritance
- Overlapping vs. disjoint
- Total vs. partial

Notation Summary

ER Diagram Symbols:

- Rectangle: Entity type
- Double rectangle: Weak entity type
- Diamond: Relationship type
- Double diamond: Identifying relationship
- Oval: Attribute
- Double oval: Multi-valued attribute
- Dashed oval: Derived attribute
- Underline: Primary key
- Dashed underline: Discriminator (partial key)
- Line: Connects entities to relationships
- Double line: Total participation
- Arrow: "One" in cardinality
- ISA triangle: Specialization/generalization

Cardinality Notations:

- Graphical: arrows and lines
- Chen: 1, N, M labels

min, max : [0,1], [1,1], [0,*], [1,*], etc.